

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Modul 2

Sensor Berdasarkan Basis Pengukuran & Aktuator Motor

Yuke Brilliant Hestiavin - 5024241016

2026

1 Pendahuluan

1.1 Latar Belakang

Sistem tertanam bekerja di antara dua dunia: program yang berjalan di mikrokontroler dan benda nyata di sekitarnya. Sensor menerjemahkan besaran fisik seperti posisi, sentuhan, medan magnet, atau jarak menjadi sinyal listrik yang bisa dibaca mikrokontroler. Aktuator melakukan hal sebaliknya, yaitu mengubah perintah listrik menjadi gerakan. Pasangan ini menjadi dasar robotika, otomasi industri, peralatan rumah pintar, dan sistem pemantauan. Hasil akhirnya bergantung pada seberapa paham kita cara sensor menghasilkan informasi dan cara aktuator menerima perintah.

Masalah yang dipelajari pada praktikum ini ada dua. Pertama, sensor dengan basis pengukuran berbeda menghasilkan jenis sinyal berbeda. Joystick menghasilkan tegangan analog. Modul sentuh, sensor Hall, dan sensor inframerah menghasilkan keluaran digital. Sensor ultrasonik menyampaikan jarak lewat lebar pulsa. Tiap jenis sinyal perlu antarmuka dan pengolahan yang sesuai, dan hasil bacanya harus diwaspadai dari derau, titik netral joystick yang bergeser, serta objek yang tidak terdeteksi.

Kedua, setiap aktuator punya cara kendali sendiri. Pada motor DC, PWM mengatur daya yang sampai ke motor. Pada servo, lebar pulsa PWM menentukan target sudut. Motor stepper membutuhkan urutan pengaktifan kumparan, dan motor brushless membutuhkan komutasi elektronik oleh ESC. Jika semuanya diperlakukan sama, motor bisa bergerak tidak sesuai perintah atau rangkaian tidak bekerja sama sekali.

Praktikum Modul 2 memakai ESP32 dengan framework Arduino untuk mempelajari seluruh rantai tersebut, dari membaca sensor, mengolah masukan, sampai menggerakkan aktuator [1]. Kemampuan ini dipakai pada sistem interaktif seperti joystick pengendali robot, sensor jarak penghindar rintangan, servo pengarah mekanisme, dan stepper untuk penempatan posisi. Setelah praktikum, praktikan diharapkan bisa memilih sensor dan penggerak sesuai kebutuhan, memahami batas ketelitian sistem, dan menyusun rangkaian serta program dengan benar.

1.2 Dasar Teori

1.2.1 Sensor, Transduksi, dan Antarmuka ESP32

Sensor bekerja dengan transduksi, yaitu mengubah suatu besaran fisik menjadi sinyal yang dapat diukur. Berdasarkan basisnya, sensor memanfaatkan perubahan resistansi, kapasitansi, induktansi, medan magnet, gelombang akustik, atau cahaya. Keluaran analog dibaca dengan ADC, keluaran digital dibaca sebagai HIGH atau LOW, dan keluaran berupa pulsa diolah dari durasi atau frekuensinya.

ADC 12 bit pada ESP32 memiliki 4096 kode, dari 0 sampai 4095. Secara ideal, untuk masukan V_{in} pada rentang 0 sampai V_{FS}, kode ADC adalah

$$k \approx \frac{V_{\text{in}}}{V_{\text{FS}}} \, (2^{12} - 1). \tag{1}$$

V_{FS} adalah batas rentang ukur yang dipakai, dan pada ESP32 nilainya dipengaruhi atenuasi serta karakteristik ADC, sehingga rumus ini tidak menggantikan kalibrasi. Resolusi hanya menunjukkan kemampuan membedakan tingkat sinyal, sedangkan akurasi juga dipengaruhi derau, linearitas, dan toleransi komponen. Pin GPIO ESP32 berlogika 3,3 V, jadi sinyal sensor 5 V harus diturunkan dulu sebelum masuk ke GPIO [1].

1.2.2 Sensor Resistif: Joystick 2-Axis KY-023

Joystick KY-023 berisi dua potensiometer untuk sumbu X dan Y, ditambah satu tombol tekan (SW). Tiap potensiometer bekerja sebagai pembagi tegangan. Jika R_1 adalah hambatan dari catu ke *wiper* dan R_2 dari *wiper* ke ground, maka

$$V_{\text{out}} = V_{\text{CC}} \frac{R_2}{R_1 + R_2}. \tag{2}$$

Menggeser tuas memindahkan *wiper*, sehingga perbandingan R_1 dan R_2 berubah dan tegangan pada VRx dan VRy berubah kontinu. Di posisi tengah ideal, tegangannya sekitar setengah catu dan kode ADC mendekati 2048. Nilai sebenarnya sering bergeser sehingga titik netral perlu diukur.

Dead zone adalah rentang di sekitar titik netral yang dianggap diam. Jika titik pusat hasil kalibrasi adalah k_0 dan toleransinya δ, kondisi $|k - k_0| < \delta$ diartikan sebagai perintah berhenti. Di luar rentang itu, tanda simpangan menentukan arah dan besarnya menentukan kecepatan. Pin SW adalah tombol terpisah yang dibaca dengan *pull-up* dan umumnya aktif LOW [1].

1.2.3 Sensor Kapasitif, Induktif, dan Efek Hall

Sensor kapasitif mendeteksi perubahan kapasitansi antara elektroda dan lingkungannya. Untuk kapasitor keping sejajar secara ideal,

$$C = \varepsilon \frac{A}{d}, \tag{3}$$

dengan ε permitivitas medium, A luas elektroda, dan d jarak antarkeping. Pada sensor sentuh, jari mengubah kopling kapasitif elektroda. IC TTP223 mengolah perubahan itu dan mengeluarkan sinyal digital, umumnya aktif HIGH saat disentuh [1]. Jadi keluaran modul adalah keputusan terdeteksi atau tidak, bukan nilai kapasitansinya.

Sensor induktif *proximity* membangkitkan medan elektromagnetik bolak-balik dengan kumparan. Benda logam di dekatnya menimbulkan arus pusar yang mengubah keadaan osilator, dan perubahan itulah yang dideteksi. Jenis ini dipakai untuk mendeteksi logam tanpa kontak.

Sensor Hall, misalnya A3144, mendeteksi medan magnet lewat efek Hall: pembawa muatan yang bergerak dalam medan magnet dibelokkan sehingga muncul tegangan melintang. IC sakelar Hall mengubah tegangan itu menjadi keluaran digital. A3144 berkeluaran *open-collector* sehingga perlu *pull-up*, dan transistor keluarannya aktif ketika medan dengan polaritas dan kekuatan yang sesuai melewati ambang [2]. Modul mengelompokkan sensor Hall bersama sensor induktif, tetapi secara fisik sensor Hall adalah sensor magnetik berbasis efek Hall dan tidak mengukur perubahan induktansi kumparan.

1.2.4 Sensor Ultrasonik HC-SR04 dan Sensor Inframerah

HC-SR04 mengukur jarak dengan prinsip *time of flight*. Setelah menerima pulsa *trigger* sekitar 10 μs, modul memancarkan delapan siklus gelombang ultrasonik sekitar 40 kHz. Lebar pulsa HIGH pada pin Echo sama dengan waktu tempuh gelombang pergi ke objek dan kembali [3]. Jaraknya

$$d = \frac{c\,t}{2}, \tag{4}$$

dengan c kecepatan suara dan t waktu tempuh pergi-pulang. Pada udara sekitar 20°C, $c \approx 343$ m/s, sehingga bentuk praktisnya $d\,[\text{cm}] \approx 0{,}0343\,t\,[\mu\text{s}]/2$. Suhu, kemiringan objek, dan sifat permukaan

memengaruhi hasil. Jika pantulan tidak kembali, pembacaan harus dianggap tidak valid lewat batas waktu (*timeout*).

Sensor IR *obstacle* memakai pemancar inframerah dan penerima pantulan. Rangkaian pembanding memutuskan apakah intensitas pantulan melewati ambang. Keluarannya hanya status deteksi, bukan jarak absolut, dan responsnya dipengaruhi warna permukaan, sudut objek, serta cahaya sekitar [1].

1.2.5 PWM dan Pengendalian Motor DC dengan H-Bridge

PWM mengatur berapa lama sinyal HIGH dalam setiap periode. Untuk periode T, waktu HIGH t_{on}, dan frekuensi f,

$$f = \frac{1}{T}, \qquad D = \frac{t_{\text{on}}}{T}, \qquad D\% = 100\% \times D. \tag{5}$$

Jika beban disambung ke catu V_s saat HIGH dan ke nol saat LOW, tegangan rata-ratanya sekitar $V \approx D\,V_s$. Pada motor DC, duty cycle yang lebih besar umumnya menaikkan kecepatan untuk beban yang sama. Hubungannya tidak selalu linear karena gesekan, beban, jatuh tegangan driver, dan GGL balik motor.

Driver H-bridge seperti L298N membalik polaritas tegangan di terminal motor untuk mengatur arah putar. Pada percobaan, IN1 dan IN2 menentukan arah dan ENA menerima PWM. GPIO hanya mengirim sinyal kendali, sedangkan energi motor datang dari catu daya motor lewat driver. Pada PWM 8 bit, nilai 0 sampai 255 mewakili duty cycle minimum sampai maksimum [1].

Program modul memakai API LEDC Arduino-ESP32 core 2.x, yaitu `ledcSetup()`, `ledcAttachPin()`, dan `ledcWrite()` berbasis *channel*. Core 3.x mengubah API ini, antara lain dengan `ledcAttach()` yang berbasis pin, sehingga versi framework harus sesuai dengan contoh kode [4].

1.2.6 Motor Servo dan Kendali Posisi

Servo posisi menggabungkan motor, roda gigi, sensor posisi internal, dan pengendali umpan balik. Pengendali membandingkan posisi aktual dengan target, lalu menggerakkan motor sampai selisihnya hilang. Pulsa kontrol dikirim sekitar 50 Hz atau berperiode 20 ms, dan lebar pulsanya menyatakan target sudut, bukan tegangan daya yang diterima motor di dalamnya.

Rentang yang sering dipakai sebagai gambaran adalah 1 sampai 2 ms, tetapi hubungan pulsa dan sudut sebenarnya mengikuti spesifikasi dan kalibrasi servo. Program praktikum memakai `attach(SERVO_PIN, 500, 2400)`, yang memetakan 500 sampai 2400 μs ke 0 sampai 180 derajat pada library. Rentang ini perlu disesuaikan dengan batas gerak servo yang dipakai [1].

1.2.7 Motor Stepper dan Driver ULN2003

Motor stepper bergerak dalam langkah diskret ketika kumparannya diaktifkan berurutan. Urutan pengaktifan menentukan arah putar, jumlah langkah menentukan sudut perpindahan, dan laju langkah menentukan kecepatan. ULN2003 adalah penguat arus untuk kumparan motor unipolar 28BYJ-48, sedangkan urutan keempat masukannya tetap ditentukan mikrokontroler.

Jika jumlah langkah per putaran poros keluaran adalah N_{rev}, sudut setelah n langkah adalah

$$\theta = n\,\frac{360^\circ}{N_{\text{rev}}}. \tag{6}$$

Modul memakai $N_{\text{rev}} = 4096$ pada pola *half-step*, sehingga 1024 langkah setara 90° secara nominal. Rasio roda gigi sebenarnya bisa membuat angka ini hanya pendekatan. Karena tidak ada umpan balik posisi, beban berlebih dapat membuat langkah hilang, dan *backlash* roda gigi membatasi ketelitian posisi [1].

1.2.8 Motor Brushless dan ESC

Motor BLDC memiliki rotor magnet permanen dan kumparan stator yang harus diberi arus bergantian secara teratur. ESC menyediakan komutasi elektronik tiga fase dengan sakelar daya. Posisi rotor bisa diketahui dari sensor atau diperkirakan dari GGL balik, tergantung jenis ESC.

Pada ESC hobi yang dipakai modul, pulsa 1000 sampai 2000 μs pada 50 Hz menyatakan throttle minimum sampai maksimum. Pulsa ini hanya perintah ke ESC, sedangkan sinyal tiga fase ke motor dibuat oleh ESC sendiri. *Arming* dilakukan dengan menahan throttle minimum sebelum motor dijalankan. Throttle nol pada program berarti pulsa minimum 1000 μs, bukan sinyal yang selalu LOW. Daya motor datang dari baterai terpisah, dan ground sinyal harus disatukan dengan ground pengendali. Pengujian dilakukan tanpa baling-baling sesuai prosedur modul [1].

2 Tugas Pendahuluan

1. **Jelaskan prinsip kerja sensor resistif seperti joystick 2-axis dalam menghasilkan nilai analog yang bervariasi sesuai posisinya.**

   Jawaban: Joystick 2-axis berisi dua potensiometer, masing-masing terhubung ke gerakan sumbu X dan sumbu Y. Kedua ujung potensiometer disambungkan ke catu dan ground, sedangkan *wiper* menjadi keluaran analog. Saat tuas digeser, *wiper* berpindah dan perbandingan resistansi pembagi tegangan berubah, sehingga tegangan VRx dan VRy mengikuti posisi tiap sumbu.

   Jika joystick dicatu 3,3 V, posisi tengah ideal memberi sekitar 1,65 V. Pada model ADC ideal dengan rentang penuh 3,3 V, tegangan itu setara kode sekitar 2048 untuk resolusi 12 bit. Nilai nyata perlu dikalibrasi karena toleransi joystick dan karakteristik ADC. Untuk mengendalikan motor, arah ditentukan oleh tanda simpangan dari titik pusat dan kecepatan oleh besarnya simpangan. *Dead zone* mencegah variasi kecil saat tuas netral membuat motor bergerak sendiri. Tombol SW dibaca sebagai masukan digital terpisah [1].

2. **Jelaskan perbedaan prinsip kerja sensor kapasitif (mis. touch sensor TTP223) dan sensor induktif (mis. hall effect sensor) dalam mendeteksi keberadaan suatu objek.**

   Jawaban: Sensor kapasitif mendeteksi perubahan kapasitansi elektroda akibat objek yang mendekat. Pada TTP223, sentuhan jari mengubah kopling kapasitif elektroda, IC menangkap perubahan itu, lalu mengeluarkan sinyal digital. Objeknya tidak harus logam. Berhasil atau tidaknya deteksi ditentukan oleh sifat dielektrik, ukuran, dan jarak objek ke elektroda [1].

   Sensor induktif yang sebenarnya memakai medan elektromagnetik dari kumparan. Logam yang mendekat menimbulkan arus pusar dan mengubah keadaan osilator, lalu rangkaian mendeteksi perubahan itu. Sensor Hall pada soal bekerja berbeda: ia membaca medan magnet lewat efek Hall. Jadi pemicu TTP223 adalah perubahan kapasitansi, pemicu sensor induktif *proximity* adalah interaksi medan dengan logam, dan pemicu sensor Hall adalah medan magnet. Sensor Hall tidak otomatis mendeteksi logam yang tidak bermagnet [2].

Pada percobaan, TTP223 diuji dengan menyentuh pad, sedangkan sensor Hall diuji dengan mendekatkan magnet. Polaritas magnet dan ambang sakelar ikut menentukan apakah keluaran Hall berubah.

3. **Bagaimana sensor ultrasonik HC-SR04 mengukur jarak? Jelaskan hubungan antara waktu tempuh pantulan gelombang suara dan jarak objek yang terdeteksi.**

   Jawaban: ESP32 mengirim pulsa *trigger* HIGH minimal sekitar 10 μs. HC-SR04 lalu memancarkan gelombang ultrasonik sekitar 40 kHz dan menunggu pantulannya. Lebar pulsa HIGH pada pin Echo dipakai sebagai waktu perjalanan gelombang pergi dan pulang [3].

   Gelombang menempuh jarak sensor ke objek dua kali, sehingga jaraknya $d = ct/2$. Sebagai contoh, untuk $t = 1000$ μs $= 0{,}001$ s dan $c = 343$ m/s:

   $$d = \frac{343 \times 0{,}001}{2} = 0{,}1715 \text{ m} = 17{,}15 \text{ cm}.$$

   Makin lama pantulan kembali, makin jauh objeknya, selama masih di dalam jangkauan sensor. Program perlu memakai *timeout* supaya tidak menunggu terus jika tidak ada pantulan. HC-SR04 umumnya dicatu 5 V, sehingga jalur Echo perlu diturunkan ke level masukan ESP32, misalnya dengan pembagi tegangan seperti pada modul [1].

4. **Jelaskan bagaimana duty cycle sinyal PWM memengaruhi kecepatan putar motor DC dan sudut motor servo, mengapa sinyal yang sama-sama berbasis PWM ini diinterpretasikan berbeda oleh kedua aktuator tersebut?**

   Jawaban: Pada motor DC, PWM menyakelar energi dari catu ke motor lewat driver. Dengan frekuensi dan catu tetap, duty cycle yang lebih besar memberi energi rata-rata lebih besar sehingga kecepatan cenderung naik pada beban yang sama. Sebagai gambaran ideal, catu 12 V pada duty cycle 25% menghasilkan tegangan rata-rata sekitar 3 V, dan pada 75% sekitar 9 V. Nilai nyatanya dipengaruhi driver, beban, dan respons motor.

   Pada servo, sinyal PWM masuk ke pengendali sebagai informasi target sudut, sementara daya motor di dalamnya datang dari jalur catu sendiri dan servo memakai umpan balik internal untuk mencapai target. Pada periode tetap 20 ms, pulsa 1 ms berarti duty cycle 5% dan pulsa 2 ms berarti 10%. Perubahan itu berarti perubahan target posisi sesuai kalibrasi servo, bukan pemberian daya motor sebesar 5% atau 10%.

   Jadi bedanya ada pada fungsi sinyal. Pada motor DC, PWM mengatur besar daya yang disakelar driver. Pada servo, lebar pulsa adalah kode perintah posisi. Jika frekuensi diubah dengan duty cycle tetap, lebar pulsa ikut berubah sehingga target servo bergeser atau sinyal menjadi tidak valid. Karena itu parameter utama kendali servo adalah lebar pulsa pada periode yang sesuai, bukan duty cycle saja [1].

5. **Apa perbedaan mendasar antara mengendalikan motor stepper dan motor DC biasa, terutama dari segi presisi posisi putaran?**

   Jawaban: Motor stepper dikendalikan lewat urutan pengaktifan kumparan. Setiap langkah menggeser rotor sebesar sudut nominal tertentu, sehingga perpindahan bisa diperintahkan dengan menghitung langkah. Pada konfigurasi modul dengan 4096 *half-step* per putaran, perintah 1024 langkah menghasilkan sudut nominal 90°. Laju langkah mengatur kecepatan, dan membalik urut-

an mengubah arah. Sebagai contoh, jeda 2000 μs antarlangkah berarti 500 langkah per detik, atau sekitar $500/4096 \times 60 \approx 7{,}3$ putaran per menit.

Motor DC biasa berputar kontinu selama diberi daya. Tegangan atau duty cycle menentukan kecenderungan kecepatannya, tetapi tidak langsung menentukan sudut akhir. Menyalakan motor selama waktu tertentu tidak menjamin perpindahan sudut yang sama, karena kecepatannya berubah menurut beban dan kondisi mekanis. Untuk mengatur posisi dengan teliti, motor DC butuh sensor posisi seperti encoder dan pengendali umpan balik.

Jadi stepper memudahkan perpindahan diskret tanpa sensor posisi pada kondisi kerja yang sesuai. Namun posisi hasil hitungan belum tentu sama dengan posisi aktual: langkah bisa hilang jika torsi kurang, roda gigi punya *backlash*, dan posisi awal perlu ditentukan, misalnya dengan *homing*. Ketelitian nyata tetap bergantung pada mekanik, beban, dan cara pengendaliannya [1].

6. **Mengapa motor brushless (BLDC) memerlukan ESC (Electronic Speed Controller) untuk dapat berputar, sedangkan motor DC biasa cukup dikendalikan melalui driver H-bridge sederhana?**

   Jawaban: Pada motor DC bersikat, sikat dan komutator memindahkan sambungan arus ke kumparan rotor secara mekanis saat poros berputar. Karena komutasi sudah dikerjakan di dalam motor, driver luar hanya perlu mengatur besar daya dan polaritas terminal. H-bridge membalik arah arus untuk mengubah arah putar, sementara PWM mengatur daya rata-rata.

   BLDC tidak punya komutator mekanis. Supaya rotor magnet permanennya terus berputar, kumparan stator harus dialiri arus dalam urutan yang sesuai dengan posisi rotor. ESC membuat urutan komutasi itu dengan sakelar daya tiga fase. Untuk BLDC tiga fase, satu H-bridge sederhana berterminal dua tidak menyediakan susunan penggerak maupun kendali komutasi yang dibutuhkan.

   ESP32 hanya mengirim perintah throttle ke ESC, lalu ESC mengubahnya menjadi penggerak tiga fase untuk motor, dengan informasi posisi rotor dari sensor atau perkiraan sesuai rancangan ESC. Pada praktikum, throttle minimum sekitar 1000 μs dikirim dulu untuk *arming*, kemudian throttle dinaikkan bertahap. Pulsa masukan ESC adalah sinyal perintah, sedangkan arus besar penggerak motor disuplai oleh rangkaian daya ESC [1].

Daftar Pustaka

[1] Laboratorium Robotika dan Sistem Cerdas, *Modul 2: Sensor Berdasarkan Basis Pengukuran & Aktuator Motor, Praktikum Sistem Mikroprosesor dan Sistem Tertanam, beserta Tugas Pendahuluan Modul 2*, `https://github.com/b401-robotics/praktikum-sismik-embed/tree/main/Modul_2`, 2026.

[2] Allegro MicroSystems, *A3141, A3142, A3143, and A3144: Sensitive Hall Effect Switches for High-Temperature Operation*, Datasheet.

[3] ElecFreaks, *Ultrasonic Ranging Module HC-SR04*, Datasheet, `https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf`.

[4] Espressif Systems, *Migration from 2.x to 3.0, dokumentasi Arduino-ESP32*, `https://docs.espressif.com/projects/arduino-esp32/en/latest/migration_guides/2.x_to_3.0.html`, Diakses: 2026.